

# 教师保留完整词元，学生高效合并：面向广告推荐中电商序列建模的 TM20K

Xinchun Li\*, Duoru Zheng\*, Wenlin Zhao\*, Haoran Ding\*, Ziyi Zhou\*, Jingxuan Tan\*, Huizhi Yang\*, Yuchen Jiang, Zhe Chen, Yuchao Zheng, Linlan Chen, Dongjian Wang, Dongyue Wang, Xiaosong Li, Hongyue Mao, Yaocheng Tan<sup>†</sup>

ByteDance

{lixinchun.bu,zhengduoru,zhaowenlin,dinghaoran,zhouziyi.828,tanjingxuan,yanghuizhi,jiangyuchen.jyc,chenzhe.john,zhengyuchao.yc,chenlinlan,wangdongjian.msg,wangdongyue,lixiaosong.1,maohongyue,tanyaocheng}@bytedance.com

## ABSTRACT

得益于超长行为序列建模，现有推荐系统能够同时考虑用户的长期兴趣与短期兴趣，从而为用户带来更好的体验。然而，序列长度的扩展给训练效率和服务吞吐量带来了沉重负担。以往方法通常采用基于搜索或基于聚类的方式压缩超长序列，但会牺牲细粒度信息；或者依赖各种轻量级目标注意力结构，却无法充分提取序列特征。本文通过结合全 Transformer 建模与两阶段知识蒸馏框架，在超长序列建模的效果与效率之间取得平衡。首先，教师模型和学生模型均采用完整注意力机制，而非纯粹的目标一序列注意力，以实现有效的序列扩展。对于学生模型，我们提出了若干简单但动机充分的词元合并方法，在保持可接受性能的同时显著压缩序列长度。随后，我们使用完整序列词元对一个一次性训练的教师模型进行充分训练，并通过知识蒸馏进一步提升学生模型的性能。所提出的范式名为 TM20K，已成功部署于字节跳动的电商广告推荐系统中，将电商序列长度扩展至 20K；在训练与服务成本几乎与线上最先进模型相同的情况下，它显著改善了关键业务指标（例如，ADSS+1.036%），同时保持了较低的服务开销（例如，服务延迟仅增加 +5.6%）。

## CCS CONCEPTS

• 信息系统 → 推荐系统; 信息检索; 计算广告.

## KEYWORDS

推荐系统，超长序列建模，词元合并，知识蒸馏

### ACM Reference Format:

Xinchun Li\*, Duoru Zheng\*, Wenlin Zhao\*, Haoran Ding\*, Ziyi Zhou\*, Jingxuan Tan\*, Huizhi Yang\*, Yuchen Jiang, Zhe Chen, Yuchao Zheng, Linlan Chen, Dongjian Wang, Dongyue Wang, Xiaosong Li, Hongyue Mao, Yaocheng Tan<sup>†</sup>. 2018. 教师保留完整词元，学生高效合并：面向广告推荐中电商序列建模的 TM20K. In *Proceedings of 请务必填写权利确认邮件中正确的会议名称 (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

\* 共同第一作者，贡献相同。

<sup>†</sup> 通讯作者。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06...\$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

## 1 引言

作为一种很有前景的规模扩展方向，超长用户行为序列建模已被广泛证明能够提升现代推荐系统的推荐质量，并通过同时捕获用户的长期与短期兴趣来改善用户满意度 [3, 10, 17, 18, 34]。然而，随着序列长度增加，训练与服务效率会急剧下降。例如，在我们的真实广告推荐系统中，将最大序列长度从 5K 扩展至 20K 时，即使为 Transformer 建模配备 FlashAttention [5] 和 M-Falcon 服务 [2, 35]，训练时间仍增加 3.5×，GPU 显存增加 49G，服务延迟约增加 6.3×（如图 1 所示）。也就是说，长序列建模的效果与效率构成了一对相互冲突的目标，需要进行恰当权衡以兼顾两者。

现有推荐系统从多个方面实现这种权衡，其中主流方案可归为两类：序列长度压缩与轻量级注意力架构。传统推荐系统广泛采用基于搜索的机制，为每个目标候选检索一部分高度相关的序列词元 [3, 18, 22, 24, 30]，但这不可避免地会丢弃细粒度且可能有价值的行为信息。此外，基于搜索的方法与 M-Falcon 服务 [2, 35] 不兼容。基于聚类的方法也可以将超长序列词元聚合为长度更短的紧凑表示 [16, 24, 29, 34, 38]，但它们通常依赖高质量的预训练词元嵌入来支持稳定聚类。仅关注目标一序列注意力，或采用各种轻量级注意力机制，可以降低 Transformer 建模的二次复杂度 [3, 9, 18, 24, 25, 31, 34]。尽管这些方法具有效率优势，此类轻量级注意力却无法充分捕获行为词元之间的内在依赖，导致序列特征提取不足。近期工作 [8] 也提出了类似观点。在我们的真实场景中，相较于轻量级目标注意力，完整 Transformer 建模确实展现出明显的性能增益。除上述端到端方案外，两阶段排序框架能够更灵活地平衡不同模型扩展方向的效果与效率，主要包括知识蒸馏 [7, 14, 15, 23] 和基础模型 [13, 17, 19, 20, 28, 32, 36] 两种范式。

在本工作中，基于初步实验研究与观察，我们采用完整注意力 (FA) 机制进行充分且有效的长序列建模，而非单纯采用目标注意力 (TA)。为解决 FA 导致的显著效率下降，我们从两个角度提出解决方案：两阶段知识蒸馏与通过词元合并进行序列长度压缩。设想图 1 所示的人类教育场景：教师通过完整、细致地通读文献来丰富知识；相比之下，学生可在教师指导下快速浏览文本，高效完成学习。受此类启发，我们将完整 Transformer 计算的繁重开销卸载给教师模型，而该模型仅训练一次，且无需在线推理。也就是说，在大规模推荐场景下，教师模型引入的开销几乎可以忽略。对于学生模型，我们精心设计了若干词元合并策略，以显著压缩序列长度，同时避免明显的性能下降。通过深入分析在线模型的注意力分数分布，我们为学生模型提出了若干简单但动机充分的词元合并策略，包括：

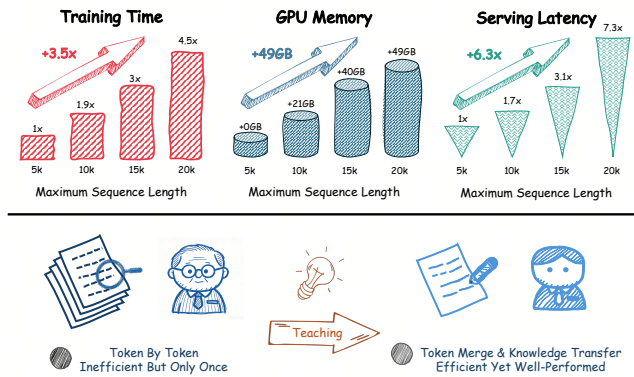


Figure 1: TM20K 的研究动机。超长序列建模会给训练时间、GPU 显存和服务延迟带来显著负担。TM20K 将繁重计算卸载到仅训练一次、保留全部原始词元的教师模型上；学生模型则借助动机充分的词元合并策略以及教师传递的知识，实现高效训练与服务。

- **局部 ID 词元合并 (LITM)**。全场景电商序列在我们的广告推荐中发挥着基础作用，它利用用户对感兴趣商品的行为<sup>1</sup>。在实践中，用户可能在较短时间窗口内与同一商品发生多次交互。我们依据商品 ID 聚合对应的序列词元。LITM 能够形成信息更集中的表示、滤除异常行为噪声，并进一步加强用户隐私保护。
- **位置自适应词元合并 (PATM)**。既有研究已证明，近期序列词元的重要性高于较早的词元 [9, 21, 30, 31]。我们的内部分析进一步表明，最近的 10% 词元约占全部注意力权重的一半。因此，我们提出一种自适应词元合并方案，根据序列词元的位置索引施加不同的合并强度。相比之下，先前工作 [2] 对整个序列均匀执行词元合并，对近期词元和历史词元赋予同等重要性。
- **分层金字塔词元合并 (LPTM)**。受大语言模型中注意力分数分析 [1, 27] 的启发，我们也观察到不同 Transformer 层具有不同的注意力模式。底层的注意力分数分布较为均匀，而高层的分数则更加集中。基于这一观察，我们在不同 Transformer 层中采用逐渐增强的词元合并策略，从而渐进式减少完整 Transformer 的计算量。

借助三种动机充分的词元合并策略，当电商序列最大长度从 5K 扩展至 20K 时，学生模型本身仅产生很小的训练与推理开销，却能带来显著的性能提升。得益于知识蒸馏，学生模型可以达到完整 20K 序列建模性能的近 85%。表 1 总结了现有针对超长序列的折中方案，其中既包括密切相关的已有工作，也包括我们提出的 TM20K-S（学生）和 TM20K-T（教师）。

总而言之，我们的主要贡献体现在以下四个方面：(a) 面向超长序列建模的新型两阶段知识蒸馏框架。一次性训练的教师模型保留完整序列词元，而学生模型可高效合并词元以支持在线服务。(b) 若干简单但动机充分的词元合并策略。所提出的三类词元合并方法易于实现，并能获得有竞争力的预测性能。(c) 显著的性能提升。大量离线实验验证了所提出 TM20K 的优势，我们的真实工业推荐系统也以可忽略的额外开销取得了显著的线上指标提升。

<sup>1</sup>所有数据均经过哈希处理，以保护用户隐私。

Table 1: 与已有工作在序列长度压缩 (LenCom) 和注意力架构 (AttnArch) 方面的比较。我们还列出了原始序列的最大长度 (MaxLen)，以及长度压缩后的有效计算序列长度 (EffLen)。详情见第 2 节。

| 方法            | MaxLen          | LenCom   | EffLen          | AttnArch  |
|---------------|-----------------|----------|-----------------|-----------|
| TWIN-V1 [3]   | $10^4$          | GSU      | $10^2$          | ETA       |
| TWIN-V2 [24]  | $10^6$          | HC & GSU | $10^2$          | ETA       |
| EST [18]      | $10^6$          | SIM      | $10^3$          | LCA & CSA |
| STCA [9]      | $10^4$          | LE       | $2 \times 10^3$ | STCA      |
| HyTRec [31]   | $10^4$          | -        | $10^4$          | HA        |
| HiSAC [34]    | $10^4$          | HC       | $2 \times 10^2$ | MHA       |
| HyFormer [11] | $3 \times 10^3$ | -        | -               | CA        |
| MTFM [25]     | -               | -        | -               | GQA & HTA |
| LONGER [2]    | $2 \times 10^3$ | TM       | $10^3$          | CA & SA   |
| TM20K-S       | $2 \times 10^4$ | TM       | $2 \times 10^3$ | FA        |
| TM20K-T       | $2 \times 10^4$ | -        | $2 \times 10^4$ | FA        |

## 2 相关工作

### 2.1 序列长度压缩

早期工作采用硬搜索或软搜索策略来选择与候选目标相关的序列物品 [22, 37]。TWIN-V1 [3] 提出保持一致性的通用搜索单元 (CP-GSU)，以提高搜索质量。TWIN-V2 [24] 进一步利用层次聚类 (HC) 对用户的终身序列进行建模。尽管这些方法支持的最大序列长度可达  $10^6$ ，但如表 1 所示，其压缩后的有效计算长度仍然相对较短。STCA [9] 遵循长度外推 (LE) 范式，在训练期间随机丢弃部分序列词元。HiSAC [34] 同样采用层次聚类 (HC)。LONGER [2] 对整个序列采用统一的词元合并策略，忽略了不同序列词元所含信息量的差异。与之不同，我们通过仔细考虑词元的属性，提出了若干动机充分的词元合并方案。

### 2.2 轻量级注意力架构

TWIN [3, 24] 采用高效目标注意力 (ETA)，而 STCA [9] 则利用堆叠式目标到历史交叉注意力 (STCA) 实现高效序列建模。EST [18] 将标准注意力分解为轻量级交叉注意力 (LCA) 和内容稀疏注意力 (CSA)。HyTRec [31] 设计了一种混合注意力架构 (HA)，将面向长期词元的线性注意力与面向用户近期交互的 softmax 注意力相结合。MTFM [25] 引入分组查询注意力 (GQA) 和定制的混合目标注意力机制，以降低计算复杂度。LONGER [2] 遵循 Perceiver 架构 [12]，进一步在底层采用交叉注意力 (CA)，并在高层采用自注意力 (SA)。值得注意的是，本文采用完整注意力，以获得更好的序列扩展能力。

### 2.3 两阶段框架

ExFM [23] 引入外部大型基础模型：它构建一个类似基础模型 (FM) 的教师模型，并让其服务于多个作为垂直模型 (VM) 的学生模型，从而摊薄构建成本。该工作启发了若干后续研究，它们采用知识蒸馏 (KD) 来提升学生模型的性能 [7, 15]。不同于这些现有方法，我们利用 KD 解决超长序列建模难题：教师模型保留完整序列词元，而学生模型实现高效训练和推理。配备各类用户嵌入或物品嵌入缓存的基础模型也已被证明能够

提升下游任务性能 [20, 32, 36]。近期的同期研究一致展示了一种新兴的扩展范式, 即实例即词元建模 [13, 17, 28]。在本工作中, 我们主要聚焦于为超长序列设计有效的知识蒸馏框架, 并将基础模型的整合作为一个有前景的未来方向。

### 3 预备知识与观察

我们首先在第 3.1 节介绍真实世界的广告推荐系统。随后, 我们在第 3.2 节比较完整注意力 (FA) 与目标注意力 (TA), 并说明设计选择。最后, 第 3.3 节报告关于注意力分布的经验观察, 这些观察为所提出的方法提供了动机。

#### 3.1 问题定义

现有工业推荐系统通常构建任务感知的用户行为序列, 以提升预测性能 [18, 30, 37]。在我们的广告推荐场景中, 用户与商品的历史交互对转化率 (CVR) 预测具有重要贡献, 可帮助判断用户是否对目标广告感兴趣。此类电商序列包含商品的固有属性 (如商品 ID、类别)、用户—物品交互特征 (如交互类型) 以及上下文信息 (如交互时间戳)。为简洁起见, 我们用  $S = \{s_1, s_2, \dots, s_L\}$  表示包含  $L$  个词元的序列。每个词元由一个关键 ID 特征  $p_i$  (如商品 ID) 及其他特征  $o_i$  构成, 形式化表示为  $s_i = [p_i, o_i]$ 。进行序列建模时, 我们首先对每个独立特征进行嵌入, 并通过求和池化聚合词元级特征:

$$E_{s_i} = \text{SumPooling}(E_{p_i}, E_{o_i}), \quad (1)$$

其中,  $E_{p_i}$ 、 $E_{o_i}$  和  $E_{s_i}$  均为  $d$  维向量。随后, 将所有词元嵌入堆叠成整体序列张量  $E_s \in \mathbb{R}^{L \times d}$ , 并将其输入序列建模模块, 以提取目标感知特征:

$$x_{\text{seq}} = \mathcal{F}_{\text{seq}}(e_t, E_s), \quad (2)$$

其中,  $e_t \in \mathbb{R}^d$  表示目标候选物品的嵌入,  $x_{\text{seq}}$  编码目标候选与历史序列词元之间的交互信息。得到序列表示后, 我们将  $x_{\text{seq}}$  与非序列特征  $x_{\text{non-seq}}$  融合, 并通过 RankMixer [39] 进行后续特征交互:

$$h = \mathcal{F}_{\text{interaction}}(x_{\text{non-seq}}, x_{\text{seq}}), \quad (3)$$

其中, 最终表示  $h$  用于预测转化率, 模型通过交叉熵损失进行更新。

#### 3.2 完整注意力与目标注意力的对比

我们开展初步研究, 以确定最优的序列建模架构  $\mathcal{F}_{\text{seq}}$ 。多数主流工作依赖注意力机制 [26], 但具体实现有所不同, 详见第 2 节。完整注意力 (FA) 的实现如下:

$$E_c = \text{Concat}(e_t, E_s), \quad (4)$$

$$A_c = \text{Softmax}\left(\frac{E_c W_Q (E_s W_K)^T}{\sqrt{d}} + M\right) E_c W_V, \quad (5)$$

$$O_c = (A_c W_u \odot \text{Swish}(A_c W_v)) W_d, \quad (6)$$

其中,  $M$  表示因果掩码, 用于禁止较早词元关注其后的词元, 这也是 M-Falcon 服务 [35] 所要求的。 $W_\star$  表示 Transformer 块的可学习参数。为简洁起见, 上述公式中省略了层归一化、多头注意力和残差连接。

相比之下, 目标注意力 (TA) 将注意力计算简化为:

$$a_t = \text{Softmax}\left(\frac{e_t W_Q (E_s W_K)^T}{\sqrt{d}}\right) E_s W_V, \quad (7)$$

$$o_t = (a_t W_u \odot \text{Swish}(a_t W_v)) W_d, \quad (8)$$

其中, 仅计算目标—序列注意力, 且只有目标特征经过 SwiGLU FFN 变换。这种固有设计带来了序列特征提取不足, 尤其是在超长序列上这一缺陷。

我们通过大量实证研究定量验证了这一局限, 如图 2 所示。我们构建了一个 6 层 Transformer 主干, 并分别采用 FA 或 TA 作为序列建模模块, 在不同最大序列长度 (5K、10K、15K 和 20K) 下进行比较。图 2(a) 报告了相对于 5K TA 基线的 AUC 增益。图 2(b) 进一步给出了在 20K 序列设置、超过半年训练数据下, FA 相对于 TA 的提升曲线。结果表明, FA 始终优于 TA, 在 20K 序列设置下实现了最高 0.25% 的 AUC 提升。值得注意的是, FA 在 5K 序列上的增益甚至比 TA 在 20K 序列上的增益高 0.15%, 这说明通过 TA 获得的规模扩展收益并不显著, 而且 FA 的性能增益绝非仅仅来自计算量增加。除预测性能外, 如图 2(c) 所示, FA 的 GPU 利用率也高于 TA。我们还在图 2(d) 中可视化了由式 5 得到的 FA 第一层注意力分布, 并分别绘制目标—序列注意力与序列—序列注意力的分布, 二者并无明显差异。对包含 5 个查询词元 (一个目标词元和四个序列词元) 及 50 个键词元的局部区域放大观察后可以发现, FA 捕获的序列—序列交互携带了不可直接舍弃的重要预测信息。这一现象与先前工作 [18] 的发现略有不同。我们推测, 其方法采用的基于搜索的序列压缩强化了目标—序列交互, 从而使序列—序列依赖相对不那么重要。通过上述分析, 我们采用 FA 机制支持有效的超长序列扩展, 并从其他优化角度追求计算效率。

#### 3.3 注意力分数分析

我们进一步对 FA 机制下一部分训练样本的注意力分数分布进行了深入分析。FA 使用完整的 20K 序列词元进行训练, 我们记录了经 Softmax (式 5) 得到的注意力概率矩阵的一个局部子矩阵。为简化起见, 我们仅报告第一个注意力头的结果。设注意力子矩阵为  $A \in \mathbb{R}^{m \times n}$ , 其中  $m$  和  $n$  分别表示查询词元数和键词元数。为降低存储开销, 我们设置  $m = 5$  和  $n = L = 20,000$ 。这  $n$  个序列词元的商品 ID 记为  $\{p_1, p_2, \dots, p_L\}$ 。

首先, 我们计算第  $t$  个查询词元在成对键词元之间的注意力分数绝对差, 即  $|A_{t,i} - A_{t,j}|$ 。我们在图 3 中绘制了这些差值, 并根据  $p_i$  是否等于  $p_j$  将其分为两组。具有相同商品 ID 的序列词元呈现出更接近的注意力分数。这一现象启发我们将具有相同 ID 的序列词元合并起来。其次, 我们按位置索引对序列词元分箱, 较小的位置索引箱对应更近期的交互词元。对于每个分箱  $[i_k, i_{k+1}]$ , 我们通过  $\sum_{j=i_k}^{i_{k+1}} A_{t,j}$  聚合注意力权重总和。每个分箱的累积注意力质量如图 3(b) 所示。最近的前 10% 序列词元贡献了近乎一半的注意力权重。这一发现促使我们开发基于词元位置的自适应词元合并策略。第三, 我们测量不同层上注意力概率分布的熵 (即  $-\sum_i A_{t,i} \log A_{t,i}$ )。图 3(c) 表明, 上层在序列词元上产生更尖锐的注意力分配, 而底层倾向于在序列词元上生成更均匀的注意力。因此, 我们尝试提出逐层渐进的词元合并方案, 以降低计算开销。

### 4 方法

本节详细介绍我们在工业广告推荐系统中为 20K 超长序列建模提出的折中方案。首先, 我们在第 4.1 节概述 TM20K 的完整流程。接着, 第 4.2 节介绍三种专为学生模型设计的、简单而有原则的词元合并策略。第三, 第 4.3 节描述一次性训练的教师模型, 并给出 KD 流程。最后, 第 4.4 节介绍若干用于提升效率或稳定训练的附加设计。

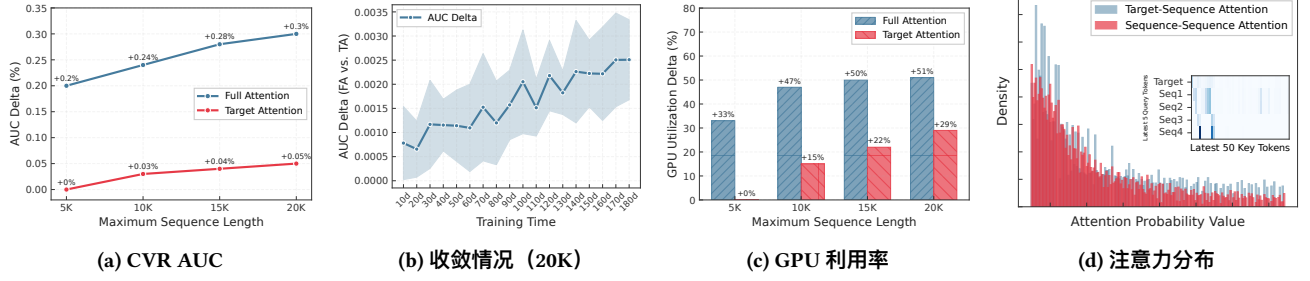


Figure 2: 从多个方面比较完整注意力 (FA) 与目标注意力 (TA)。

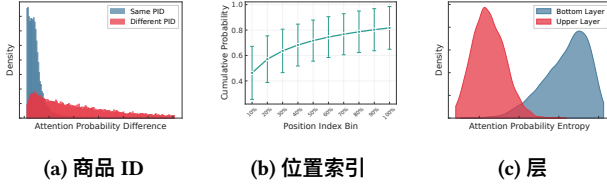


Figure 3: 基于部分训练样本计算得到的注意力分数统计与可视化。

#### 4.1 TM20K 框架

我们提出的 TM20K 整体框架如图 4 所示。我们遵循知识蒸馏 (KD) 框架 [7, 15, 23], 以实现更灵活的超长序列扩展。教师模型与学生模型均采用基于完整注意力 (FA) 的 Transformer 建模, 以捕获全面的序列信息。如第 3.1 节所述, 我们的模型架构利用 RankMixer [39] 对序列表示与非序列表示进行高阶交互。教师模型保留完整序列词元, 不进行任何序列压缩。尽管效率较低, 教师模型只需训练一次, 且无需部署在线推理。相比之下, 负责在线服务的学生模型通过三类词元合并方法处理压缩后的序列, 包括: (1) 局部 ID 词元合并 (LITM); (2) 位置自适应词元合并 (PATM); (3) 分层金字塔词元合并 (LPTM)。

##### Algorithm 1 局部 ID 词元合并 (LITM)

**Require:**  $E \in \mathbb{R}^{L \times d}$ ,  $p \in \mathbb{I}^L$ ,  $T$ : 最大间隔阈值

- 1: 初始化  $p2i \leftarrow \emptyset$ ,  $G \leftarrow \emptyset$  ID 到索引的映射; 临时合并列表
- 2: **for**  $j = 0$  到  $L - 1$  **do**
- 3:   **if**  $p_j \notin p2i$  **或**  $j - p2i[p_j] > T$  **then**
- 4:      $G.append(E_j)$  ▷ 添加新词元
- 5:      $p2i[p_j] \leftarrow |G| - 1$  ▷ 记录索引
- 6:   **else**
- 7:      $G[p2i[p_j]] \leftarrow \text{Sum}(G[p2i[p_j]], E_j)$  ▷ 合并词元
- 8:   **end if**
- 9: **end for**
- 10: **return**  $E_m \in \mathbb{R}^{L' \times d} \leftarrow G$  ▷ 合并后的序列

#### 4.2 TM20K: 学生模型

受注意力分数实证观察 (第 3.3 节) 的启发, 我们为学生模型设计了三种动机充分的词元合并方法, 并给出了具体实现。

**4.2.1 局部基于 ID 的词元合并 (LITM)**. 给定初始序列词元嵌入  $E \in \mathbb{R}^{L \times d}$  和键 ID (例如商品 ID) 列表  $p \in \mathbb{I}^L$ .  $\mathbb{I}$  是包含所有

唯一 ID 的离散集合。我们先前观察到, 具有相同 ID 的序列词元的注意力分数更为接近, 这启发我们将这些词元合并起来。具体而言, 我们提出局部基于 ID 的词元合并 (LITM), 它连续合并具有相同 ID 的词元。伪代码总结于算法 1。其中,  $T$  作为超参数, 定义了合并具有相同键 ID 的词元时所允许的最大位置间隔。在实现中, 我们将  $T = 3$  固定, 以聚合短局部窗口内针对同一键 ID 的连续用户交互。

**4.2.2 按位置自适应词元合并 (PATM)**. PATM 是一种按位置执行的词元合并方案, 它在先前注意力分析的指导下, 对不同位置的词元应用不同的压缩强度。它保留近期词元的详细信息, 同时大幅压缩较旧且信息量较低的词元。我们首先将长度为  $L$  的序列划分为  $B$  个互不相交的分段, 这些分段由范围列表  $R$  定义, 其中每个元素  $R_b$  都是包含起始和结束位置的元组。我们对每个分段应用 [2] 中提出的词元合并操作, 各分段特定的压缩因子存储在向量  $K$  中。伪代码列于算法 2。需要注意的是, 我们假设位置索引较小的词元对应更近期的用户交互。因此, 分段范围和压缩因子只需满足核心原则: 近期行为尽量少地进行词元合并, 而较旧的历史交互则进行更激进的压缩。例如, 给定  $L = 20,000$ ,  $B = 4$ , 分段范围  $R = [[1, 1, 000], [1, 001, 5, 000], [5, 001, 10, 000], [10, 001, 20, 000]]$  和压缩因子  $K = [1, 2, 3, 4]$ , 合并后的序列长度约为 7,000。一个实现细节是, 我们使用全零词元填充分段, 以确保其长度能被相应的词元合并因子整除。

##### Algorithm 2 按位置自适应词元合并 (PATM)

**Require:**  $E \in \mathbb{R}^{L \times d}$ ,  $B, R, K$

- 1: 初始化  $G \leftarrow \emptyset$  ▷ 临时合并列表
- 2: **for**  $b = 0$  到  $B - 1$  **do**
- 3:    $(s, e) \leftarrow R_b$  ▷ 分段  $b$  的起始和结束位置
- 4:    $k \leftarrow K_b$  ▷ 当前分段的压缩因子
- 5:    $\hat{E} = \text{Sum}(\text{Reshape}(E[s : e], (-1, k, d)), \text{dim} = 1)$  ▷ 合并  $k$  个连续词元
- 6:    $G.extend(\hat{E})$
- 7: **end for**
- 8: **return**  $E_m \in \mathbb{R}^{L' \times d} \leftarrow G$  ▷ 合并后的序列

**4.2.3 逐层金字塔词元合并 (LPTM)**. LPTM 通过逐层渐进的词元合并策略, 降低 FA 带来的计算开销。多层之间的序列长度形成金字塔结构, 其中底层使用较长的序列, 而上层处理短得多的序列。假设第  $n$  个 Transformer 层的输出为  $\hat{E}_n \in \mathbb{R}^{L_n \times d}$ ,

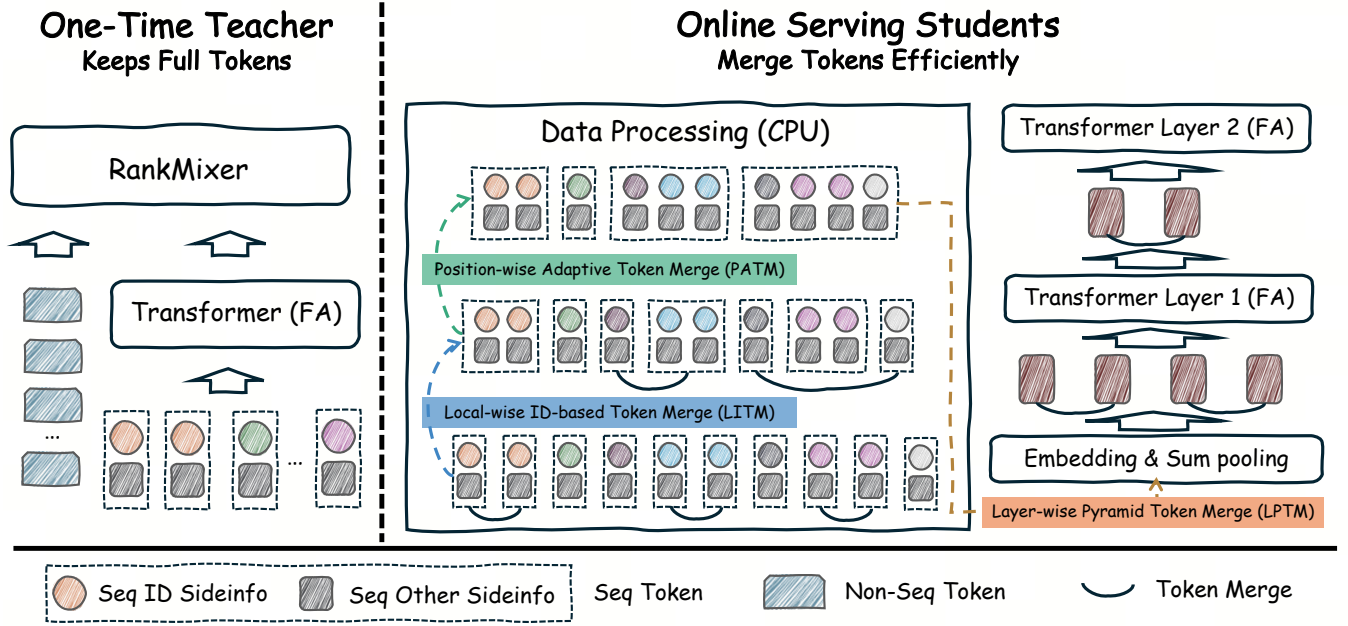


Figure 4: TM20K 的整体框架。我们的模型使用全注意力 (FA) 对序列建模, 并使用 RankMixer 进行异构词元交互。教师模型在不采用任何词元合并策略的情况下学习完整词元, 而学生模型则采用三种简单且动机充分的词元合并策略, 包括: (1) 局部基于 ID 的词元合并 (LITM), (2) 按位置自适应词元合并 (PATM), 以及 (3) 逐层金字塔词元合并 (LPTM)。

则第  $(n+1)$  层的输入按以下方式合并:

$$E_{n+1} = \text{Sum} \left( \text{Reshape} \left( \hat{E}_n, (L_n/2, 2, d) \right), \text{dim} = 1 \right), \quad (9)$$

其中,  $E_{n+1} \in \mathbb{R}^{L_n/2, d}$  大幅降低了后续注意力层的计算成本。伪代码如算法 3 所示, 其中  $N$  表示 Transformer 层数。只有历史序列词元会进行逐层词元合并, 目标候选词元永不会被合并。为简洁起见, 我们在伪代码中省略了这一实现。此外, 式 9 中定义的操作可以每隔若干层执行一次, 以平衡效果和效率。在我们的场景中, 使用 6 层 Transformer, 并每 2 层执行一次该操作。

#### Algorithm 3 逐层金字塔词元合并 (LPTM)

**Require:**  $E_0 \in \mathbb{R}^{L_0 \times d}$ ,  $N$

```

1: for  $n = 0$  到  $N - 1$  do
2:    $\hat{E}_n \leftarrow \text{TransLayer}(E_n)$             $\triangleright$  根据式 4-式 6
3:    $E_{n+1} \leftarrow \text{Merge}(\hat{E}_n)$             $\triangleright$  根据式 9
4: end for
5: return  $E_N \in \mathbb{R}^{L' \times d}$ 

```

**4.2.4 工程实现.** 在我们的工业广告推荐系统中, 我们依次应用 LITM、PATM 和 LPTM 压缩 20K 序列, 从而显著降低学生模型的计算成本。我们在 CPU 上实现 LITM 和 PATM 的计算。首先, 这些基于规则的词元合并方案适合 CPU, 却不适于 GPU 加速。此外, 我们避免在 CPU 上合并并嵌入, 而是合并每个序列词元的原始特征。通过 LITM 和 PATM 对词元特征分组后, 我们从嵌入表中查找每个词元的特征嵌入, 随后执行求和池化。这一设计显著降低了 CPU 与 GPU 之间的通信带宽开销。对于

LPTM, 它仅需对标准 Transformer 架构做少量修改, 式 9 所定义的合并操作可通过 GPU 上的张量运算高效实现。

### 4.3 TM20K: 教师模型

由于教师模型只训练一次, 我们对其效率并不过分关注。关键的是, 教师模型独立于所有学生模型进行训练, 其预测的 *logit* 会被缓存, 用于监督学生模型训练。教师模型遵循第 3.1 节介绍的架构。设教师模型的预测 *logit* 为  $g_T = \text{MLP}(h_T)$ , 其中  $h_T \in \mathbb{R}^d$  表示式 3 中获得的最终表示。随后, 教师模型通过针对真实标签  $y \in \{0, 1\}$  的二元交叉熵损失进行优化:

$$\ell_{ce} = -y \log q_T - (1 - y) \log(1 - q_T), \quad (10)$$

其中,  $q_T = \text{Sigmoid}(g_T)$  是预测转化率, 并会缓存用于 KD。

每个学生模型均使用交叉熵损失和蒸馏损失进行联合优化。遵循先前的工作 [7, 23], 我们为学生模型设计了两个不同的预测头, 包括一个主塔和一个蒸馏塔, 形式化如下:

$$\ell_{\text{main}} = -y \log q_{S, \text{main}} - (1 - y) \log(1 - q_{S, \text{main}}), \quad (11)$$

$$\ell_{\text{dis}} = \ell_{ce} + \lambda \ell_{kd}, \quad (12)$$

$$\ell_{ce} = -y \log q_{S, \text{dis}} - (1 - y) \log(1 - q_{S, \text{dis}}), \quad (13)$$

$$\ell_{kd} = -q_T \log q_{S, \text{dis}} - (1 - q_T) \log(1 - q_{S, \text{dis}}). \quad (14)$$

其中,  $q_{S, \star} = \text{Sigmoid}(\text{MLP}_{\star}(h_S))$  与  $\star \in \{\text{main}, \text{dis}\}$  分别表示主塔和蒸馏塔预测的转化概率。 $\lambda$  是一个超参数。

### 4.4 其他设计

我们的基线模型在训练与推理过程中采用了一系列优化工具, 包括用户级训练 (类似于 [9] 中的 RLB)、FlashAttention [5]、混

**Table 2: 所有方法的整体性能与效率对比。报告了最大序列长度 (MaxL)、长度压缩策略 (LenCom)、实际计算序列的平均长度和第 90 百分位长度 (AvgL 和 P90L)。所有 AUC/LogLoss 差值均相对于已长期在线服务的 5K 基线模型计算。**

| 最大长度 | 长度压缩            | 平均长度        | P90 长度      | 方法       | AUC(↑) | ΔAUC(%)       | LogLoss(↓) | ΔLogLoss(%)   | 吞吐量        | 显存  |
|------|-----------------|-------------|-------------|----------|--------|---------------|------------|---------------|------------|-----|
| 5K   | LITM0<br>& UTM2 | 1.2K        | 1.6K        | Base     | 0.8212 | –             | 0.4917     | –             | 88K        | 52G |
|      |                 |             |             | TWIN     | 0.8159 | -0.65%        | 0.4984     | +1.37%        | 200K       | 30G |
|      |                 |             |             | STCA     | 0.8200 | -0.15%        | 0.4931     | +0.29%        | 137K       | 40G |
|      |                 |             |             | HiSAC    | 0.8207 | -0.06%        | 0.4922     | +0.11%        | 114K       | 47G |
|      |                 |             |             | LONGER   | 0.8215 | +0.04%        | 0.4914     | -0.07%        | 100K       | 58G |
|      |                 |             |             | MTFM     | 0.8213 | +0.01%        | 0.4916     | -0.02%        | 105K       | 37G |
|      |                 |             |             | HyFormer | 0.8216 | +0.05%        | 0.4913     | -0.08%        | 80K        | 55G |
| 20K  | UTM2            | 4.4K        | 10K         | TWIN     | 0.8165 | -0.57%        | 0.4967     | +1.02%        | 102K       | 53G |
|      |                 |             |             | STCA     | 0.8210 | -0.02%        | 0.4920     | +0.05%        | 58K        | 68G |
|      |                 |             |             | HiSAC    | 0.8214 | +0.03%        | 0.4915     | -0.05%        | 15K        | 63G |
|      |                 |             |             | LONGER   | 0.8221 | +0.11%        | 0.4907     | -0.21%        | 48K        | 70G |
|      |                 |             |             | MTFM     | 0.8219 | +0.08%        | 0.4909     | -0.16%        | 27K        | 81G |
|      |                 |             |             | HyFormer | 0.8222 | +0.12%        | 0.4906     | -0.23%        | 36K        | 75G |
| 20K  | TM20K           | <b>8.8K</b> | <b>20K</b>  | T        | 0.8233 | <b>+0.26%</b> | 0.4890     | <b>-0.55%</b> | <b>11K</b> | 86G |
| 20K  | TM20K           | <b>1.8K</b> | <b>2.6K</b> | S        | 0.8224 | <b>+0.15%</b> | 0.4903     | <b>-0.29%</b> | <b>83K</b> | 74G |
| 20K  | TM20K           | <b>1.8K</b> | <b>2.6K</b> | S w/ KD  | 0.8230 | <b>+0.22%</b> | 0.4896     | <b>-0.43%</b> | <b>83K</b> | 74G |

合精度训练 [2]、去除填充以及 M-Falcon 服务 [35]。此外，我们还引入以下两项优化：

- **序列堆叠。**同一批次内的序列长度非常不均衡。原先在执行嵌入表查找时，所有序列都会被填充到与最长序列一致的长度，从而得到嵌入张量  $E_s \in \mathbb{R}^{B \times L_{\max} \times d}$ 。实践中，有效词元总数  $L_{\text{total}} = \sum_{i=1}^B L_{\text{valid},i}$  往往远小于  $B \times L_{\max}$ ，造成大量 GPU 显存浪费。为缓解这一问题，我们将所有有效词元均匀地重新分配到批次内各实例中，并记录每个实例的准确有效词元数。由此得到的嵌入张量变为  $E_{\text{stack},s} \in \mathbb{R}^{B \times \bar{L} \times d}$ ，其中  $\bar{L} = \lceil L_{\text{total}}/B \rceil$ 。随后，我们依据记录的词元数量在 GPU 上重建去除填充后的张量。该技术可显著降低超长序列的显存占用。例如，对完整 20K 序列而言，它最多可减少 10 GB 的 GPU 显存使用量。
- **QK 归一化。**引入超长序列建模和 KD 损失后，我们观察到学生模型训练不稳定，常常出现训练发散。借鉴 LLM 中为缓解极端注意力分数所致不稳定性而广泛采用的方案 [6]，我们在计算注意力之前对查询词元和键词元施加归一化层。尽管这会带来很小的 GPU 显存与计算开销，但它能够有效稳定整个训练过程。

## 5 实验

本节首先介绍实验设置，随后给出整体比较结果，并呈现有关 TM20K 所提方法的消融研究。最后，我们报告真实广告推荐系统中的线上 A/B 测试结果。

### 5.1 实验设置

**5.1.1 数据集。**我们在从真实广告系统收集的工业规模 CVR 预测数据集上进行实验。训练语料跨越连续两个月，共包含数十亿个训练样本。超长用户序列由历史电子商务交互构成。除序列特征外，我们还将用户特征、物品特征和上下文特征输入传

统特征交互模块进行进一步建模。所有训练数据均通过删除敏感的用户/物品信息并对特征 ID 进行哈希处理而匿名化。

**5.1.2 基线。**基线模型采用配备全注意力 (FA) 的 6 层 Transformer，对用户的电子商务历史序列进行建模，最大输入长度为 5K。我们的核心目标是将支持的最大序列长度从 5K 扩展到 20K，力求在不引入过高计算开销的情况下取得显著的预测性能提升。我们将 TM20K 框架与以下最先进的高效序列模型进行比较。STCA [9] 利用堆叠交叉注意力和长度外推来提升训练效率。LONGER [2] 将固定均匀词元合并 (UTM) 用于序列压缩，并结合 Perceiver [12] 架构。MTFM [25] 采用分组查询注意力和混合目标注意力来降低序列建模复杂度。HyFormer [11] 设计了专用交互块，对序列特征与非序列特征之间的交叉关系进行建模。我们还运行了经典的基于搜索和基于聚类的序列推荐方法，包括 TWIN [3] 和 HiSAC [34]。

**5.1.3 超参数与评估指标。**Transformer 的隐藏维度为 512。我们采用带 SwiGLU 激活的前馈网络 (FFN)，FFN 中间维度设为 1024。全局训练批大小固定为 320。由于 GPU 内存负担，在使用完整 20K 序列词元训练教师模型时，我们使用较小的批大小 %。所有实验均运行于拥有数百块 GPU 的分布式 GPU 集群。我们使用下一批次评估机制计算预测指标。我们通过报告预测性能和计算效率指标，对所有方法进行全面评估。ROC 曲线下面积 (AUC) 和 LogLoss 作为预测评估指标。训练吞吐量 (Thr) 和 GPU 峰值内存使用量 (Mem) 用于衡量计算效率。我们还报告了每种训练配置下已处理输入序列的三个统计指标，包括最大序列长度 (MaxLen)、平均序列长度 (AvgLen) 和第 90 百分位序列长度 (P90 Len)。由于模型评估使用的数据量很大，我们观察到重复运行模型时的 AUC 波动在 0.01% 以内。因此，所有报告的实验结果均省略标准差。

## 5.2 性能对比

我们进行了全面比较, 覆盖 MaxLen 为 5K 的基线序列和 MaxLen 为 20K 的扩展序列, 并报告所有方法的性能与效率指标。基线序列采用简化版的基于 ID 的词元合并 (记为 LITM0) 和  $K = 2$  的均匀词元合并 (记为 UTM2) [2] 处理 5K 序列。结果列于表 2。TWIN [3] 由于其序列建模范式过于简单 (例如只有一个交互层), 导致性能明显下降。

在 5K 长度设置下, STCA 的预测性能较差, AUC 下降 0.15%。在 Transformer 深度和隐藏维度配置相同的情况下, 堆叠交叉注意力的建模能力无法匹敌标准全注意力。其他三种有竞争力的长序列方法分别取得 +0.04%、+0.01% 和 +0.05% 的小幅精度提升。在效率方面, STCA 和 MTFM 减少了 GPU 内存使用量并提高了训练吞吐量, 而 HyFormer 以吞吐量下降和内存开销增大为代价, 带来了小幅精度改善。当将最大序列长度扩展到 20K 时, 我们对所有对比方法采用 UTM2 压缩, 因为使用原始完整长度的 20K 序列进行训练会导致 GPU 内存爆炸。HiSAC [34] 依赖充分训练的多模态嵌入来支持聚类流程, 带来了小幅性能改善。除 STCA 外, 其他三种对比方法获得了 0.08% 至 0.12% 的正向 AUC 增益, 证明更长的用户行为序列携带了额外的预测信号。然而, 效率急剧恶化, 所有模型的吞吐量较 5K 基线模型均下降超过 50%, 限制了它们的实际在线部署。

在我们提出的 TM20K 框架中, 教师模型 TM20K-T 取得了最大的 AUC 提升 +0.26% 和最低的 LogLoss (-0.55%), 证实完整且未压缩的 20K 序列能获得最优预测质量。然而, 其吞吐量从基线的 88K 骤降至仅 11K, 同时 GPU 内存使用量很高, 因而该教师模型不适合在线推理。得益于我们提出的 LITM、PATM 和 LPTM 三种词元合并方法, 学生模型 TM20K-S 将平均序列长度从教师模型的 8.8K 缩短到仅 1.8K。其单独使用时的 AUC 增益适度下降至 +0.15%, 但吞吐量仅小幅下降至 83K, 与 5K 基线模型相比, 吞吐量仅下降 5%。我们的词元合并策略大幅缩小了长序列学生模型与原始短序列基线之间的效率差距。引入来自高质量教师 logit 的 KD 后, 采用 KD 的学生模型 TM20K-S 获得了显著的预测性能提升。其 AUC 增益上升至 +0.22%, 恢复了教师模型总性能提升的约 85%。预期进一步增强教师模型的能力将为学生模型带来额外收益。

**Table 3: 词元合并策略的消融研究。结果均相对于对应的完整序列基线。**

| TM 策略               | AvgL | P90L  | $\Delta$ AUC( $\uparrow$ ) | Thr |
|---------------------|------|-------|----------------------------|-----|
| 5K Full             | 4K   | 5K    | –                          | 40K |
| +UTM2               | 2K   | 2.5K  | -0.03%                     | 72K |
| +LITM0 & UTM2       | 1.2K | 1.6K  | -0.05%                     | 88K |
| 20K Full            | 8.8K | 20K   | –                          | 11K |
| +RandDrop           | 5.3K | 13.7K | -0.11%                     | 21K |
| +RandTrunc          | 5.3K | 13.5K | -0.10%                     | 21K |
| +LITM               | 5K   | 11.3K | -0.02%                     | 25K |
| +PATM               | 4.5K | 10.2K | -0.06%                     | 34K |
| +LPTM               | 8.8K | 20K   | -0.03%                     | 19K |
| +LITM & PATM        | 1.8K | 2.6K  | -0.07%                     | 68K |
| +LITM & PATM & LPTM | 1.8K | 2.6K  | -0.11%                     | 83K |

## 5.3 消融研究

**5.3.1 词元合并策略的消融实验。**我们开展消融实验, 以量化 LITM、PATM 和 LPTM 对预测性能与训练效率的单独及组合影响, 结果汇总于表 3。首先, 我们以 5K 完整序列基线为基础, 依次启用 UTM2 和 LITM0 压缩来评估不同变体。仅应用 UTM2 会使 AUC 轻微下降 0.03%, 但训练吞吐量从 40K 提升至 72K。在 UTM2 之上叠加 LITM0 后, 平均序列长度被压缩至 1.2K。整体 AUC 仅下降 0.05%, 而吞吐量进一步提高至 88K。随后, 我们转向 20K 超长序列设置, 并以未压缩的 20K 完整序列基线为参照, 评测各个所提出的词元合并模块。LITM 将平均序列长度从 8.8K 缩短到 5K (减少 40%), PATM 则将其压缩至 4.5K (减少 48%)。LPTM 不改变整体序列长度分布, 但仅带来 0.03% 的小幅 AUC 下降, 并显著提升吞吐量。需要指出的是, 我们只报告第一输入层的序列长度; 借助 LPTM, 中间层处理的序列更短。在效率方面, 相较于 20K 完整序列基线, LITM 使吞吐量接近翻倍 ( $+1\times$ ), PATM 则使吞吐量达到约三倍 ( $+2\times$ )。在预测性能方面, 单独采用 LITM、PATM 和 LPTM 分别导致 0.02%、0.06% 和 0.03% 的 AUC 下降。当三种策略全部集成时, 累计 AUC 损失达到 0.11%。也就是说, 引入的词元合并策略不可避免地在梯度更新中产生冲突, 从而导致性能下降。不过, 这些策略均得到注意力分数分布实证观察的充分支持, 并已尽可能降低信息损失。相比之下, 若采用与位置无关的随机丢弃 (RandDrop) 或随机尾部截断 (RandTrunc), 随机移除约 40% 的序列词元, AUC 损失已达到约  $-0.1\%$ 。

**Table 4: LITM 和 PATM 的超参数分析。结果均相对于完整 20K 序列。**

|      | 超参数                   | $\Delta$ AvgLen( $\downarrow$ ) | $\Delta$ AUC( $\uparrow$ ) |
|------|-----------------------|---------------------------------|----------------------------|
| LITM | $T = 1$               | -39%                            | -0.02%                     |
|      | $T = 3$               | -43%                            | -0.02%                     |
|      | $T = 10$              | -57%                            | -0.04%                     |
| PATM | 2K:1,4K:2,8K:3,20K:4  | -55%                            | -0.09%                     |
|      | 3K:1,5K:2,10K:3,20K:4 | -49%                            | -0.06%                     |
|      | 4K:1,8K:2,12K:3,20K:4 | -40%                            | -0.05%                     |

**5.3.2 词元合并超参数的消融实验。**我们对 LITM 和 PATM 进行超参数敏感性实验, 以探索序列压缩率与预测性能之间的权衡, 详细结果列于表 4。对于 LITM, 我们在 1、3 和 10 之间调节间隔阈值  $T$ 。当  $T$  从 1 增大到 3 时, 平均序列长度额外压缩了 4%, 而 AUC 保持不变。对于 PATM, 我们测试了三组分段范围及其对应的压缩因子。为了便于表示, 配置 “2K:1,4K:2,8K:3,20K:4” 表示我们根据 2K、4K 和 8K 三个分界点, 将完整的 20K 序列划分为四个分段。随后, 对每个分段执行词元合并, 合并因子分别设置为 1、2、3 和 4。可以看到一个清晰的正相关关系: 更激进的序列压缩会持续导致更大的 AUC 下降。对于工业在线部署, 我们选择了一种中等压缩程度的 PATM 配置。我们的调参原则优先将在线服务延迟控制在可接受范围内, 即使可以容忍稍高的 AUC 损失, 也要避免效率过度恶化。

**5.3.3 蒸馏权重的消融研究。**我们研究式 12 中  $\lambda$  的不同取值对学生模型 AUC 的影响。详细结果汇总于表 5。当  $\lambda$  设为 50 时, 模型取得最佳预测性能。进一步检查损失量级可以发现, 在  $\lambda = 50$  的条件下, 缩放后的蒸馏损失项  $\lambda \ell_{kd}$  与损失  $\ell_{ce}$  的量级几乎完全相同。

Table 5: 采用不同蒸馏权重时，相对于 TM20K-S 模型的 AUC 增益。

|                        | $\lambda=30$ | $\lambda = 50$ | $\lambda=75$ | $\lambda=100$ | $\lambda=150$ |
|------------------------|--------------|----------------|--------------|---------------|---------------|
| $\Delta AUC(\uparrow)$ | -0.01%       | <b>+0.00%</b>  | -0.01%       | -0.01%        | -0.03%        |

Table 6: 相对于 TM20K-S 模型的其他实现细节结果。

|                       | 早期 $\Delta AUC(\uparrow)$ | 后期 $\Delta AUC(\uparrow)$ | Thr |
|-----------------------|---------------------------|---------------------------|-----|
| w/ MeanPooling & LogN | -0.01%                    | -0.01%                    | +0% |
| w/ PATM on SeqInc     | +0.01%                    | +0.01%                    | +0% |
| w/o QK Norm           | +0.00%                    | -0.45%                    | +2% |
| w/o KD in Late Period | +0.00%                    | +0.00%                    | +0% |

5.3.4 其他实现细节的消融研究. 我们开展了一些辅助实验, 结果见表 6。首先, 我们在词元合并中以平均池化替代求和池化。遵循已有工作 [24], 我们将  $\log(n)$  引入注意力权重, 其中  $n$  表示被合并的词元数量。然而, 这一修改导致 AUC 下降 0.01%。其次, 当序列长度从 5K 扩展到 20K 时, 我们将 PATM 压缩仅应用于超出原始 5K 序列的增量词元。该设置使 AUC 小幅提升 0.01%, 但显著增加了模型开发的工程复杂度。第三, 我们评测了不使用 QK 归一化进行训练时的性能与效率 [6]。早期预测性能基本不变, 但随着训练推进可能出现训练发散。最后, 我们在训练后期移除蒸馏损失项。最终预测性能几乎保持不变, 这表明经过足够次数的蒸馏训练后, 学生模型已充分吸收教师模型的知识。

Table 7: 工业广告场景中的线上 A/B 测试结果。

|               | ADSS         | ADV V            | 延迟    |
|---------------|--------------|------------------|-------|
| TM20K-S       | +0.881% (0%) | +0.515% (0.004%) | +5.6% |
| TM20K-S w/ KD | +1.036% (0%) | +0.780% (0%)     | +5.6% |

5.4 线上 A/B 测试结果

将 TM20K 框架部署到工业广告推荐系统后, 核心业务指标——广告主得分 (ADSS) 和广告主价值 (ADV V) 均取得了可衡量的提升。我们为线上实验分配总体流量的 10%, 实验持续五天, 服务数亿用户。所有线上 A/B 测试结果汇总于表 7。基线是一个成熟的高性能广告排序模型, 该模型已长期在线部署, 并对 5K 电商序列进行建模。TM20K 在所有业务指标上均取得具有统计显著性的正向增益, 其中 ADSS 提升 1.036%, 线上服务延迟仅小幅增加 5.6%。A/B 测试的 p 值列于表 7 的括号中, 证实所观察到的增益具有统计显著性。值得注意的是, 教师模型已部署长达 8 个月, 为数十项学生模型研究提供蒸馏信号, 并且迄今从未重新训练。我们认为, 教师模型每年重新训练一次的成本, 摊销到所有模型训练周期后相对较低。此外, 在线服务期间我们采用显存友好的 M-Falcon [35] 策略, 使所有目标候选共享相同的序列信息。因此, 服务成本主要来自延迟增加所需的额外 GPU 卡 (即 5.6%), 但与线上收益增量相比, 该成本可以忽略不计。

6 优势与局限性

我们提出的 TM20K 框架具有以下优势: (a) 所设计的词元合并策略易于开发和实现, 且其设计由注意力分数分析所得的可靠观察指导; (b) 不同于依赖词元检索或长度外推的已有方法, TM20K 无论在教师模型还是学生模型中都不丢弃任何词元; (c) 我们验证了两阶段蒸馏框架能够在工业级大规模推荐系统中实现超长序列扩展, 在保持良好效率的同时取得显著业务提升。

然而, 本工作仍存在局限。具体而言, 所提出的词元合并方案本质上是基于规则的。尽管我们针对广告场景进行了详尽的消融研究和超参数分析, 但跨领域部署仍可能需要额外调参, 以适配相应应用场景。此外, 我们尚未在现有场景中引入稀疏注意力机制 [4, 8, 33]。要让这些稀疏注意力实现与用户级训练、去除填充和 FlashAttention [5] 等现有优化兼容, 需要开发高度定制化的 GPU 算子。这会带来巨大的工程开销, 因此我们将此类稀疏注意力方案的探索留待未来工作。

7 结论

本文提出 TM20K, 旨在工业广告推荐系统的超长序列建模中平衡预测性能与计算效率。对注意力分数的全面分析验证了 Transformer 建模采用完整注意力 (FA) 的必要性。为进一步缓解 FA 引入的计算复杂度, TM20K 采用两阶段知识蒸馏范式。教师模型基于完整序列词元进行训练, 而学生模型利用三种动机充分、精心设计的词元合并策略压缩超长序列。通过将用户电商行为序列的最大长度从 5K 扩展至 20K, TM20K 以可忽略的额外在线服务延迟, 实现了具有统计显著性的 +1.036% ADSS 提升。

## REFERENCES

- [1] Amit Ben Artzy and Roy Schwartz. 2024. Attend first, consolidate later: On the importance of attention in different llm layers. In *Proceedings of the 7th Black-box NLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. 177–184.
- [2] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. Longer: Scaling up long sequence modeling in industrial recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 247–256.
- [3] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, et al. 2023. TWIN: Two-stage interest network for lifelong user behavior modeling in CTR prediction at kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3785–3794.
- [4] Chenglong Chu, Guorui Zhou, Guowang Zhang, Han Li, Hao Peng, Hongtao Cheng, Jian Liang, Jiangxia Cao, Kun Gai, Lingzhi Zhou, et al. 2026. Kwai Summary Attention Technical Report. *arXiv preprint arXiv:2604.24432* (2026).
- [5] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [6] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. 2023. Scaling vision transformers to 22 billion parameters. In *International conference on machine learning*. PMLR, 7480–7512.
- [7] Haoran Ding, Wenlin Zhao, Yuchen Jiang, Juren Li, Jie Zhu, Xinchun Li, Yishujie Zhao, Yi Zhang, Ao Qiao, Jianhui Dong, et al. 2026. Rec-Distill: An Industrial Distillation Pipeline for Large-Scale Recommendation Models. *arXiv preprint arXiv:2605.29755* (2026).
- [8] Qin Ding, Kevin Course, Linjian Ma, Jianhui Sun, Ruochen Liu, Zhao Zhu, Chunxing Yin, Wei Li, Dai Li, Yu Shi, et al. 2026. Bending the scaling law curve in large-scale recommendation systems. *arXiv preprint arXiv:2602.16986* (2026).
- [9] Lin Guan, Jia-Qi Yang, Zhishan Zhao, Beichuan Zhang, Bo Sun, Xuanyuan Luo, Jinan Ni, Xiaowen Li, Yuhang Qi, Zhifang Fan, et al. 2025. Make It Long, Keep It Fast: End-to-End 10k-Sequence Modeling at Billion Scale on Douyin. *arXiv preprint arXiv:2511.06077* (2025).
- [10] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 585–593.
- [11] Yunwen Huang, Shiyong Hong, Xijun Xiao, Jinqui Jin, Xuanyuan Luo, Zhe Wang, Zheng Chai, Shikang Wu, Yuchao Zheng, and Jingjian Lin. 2026. HyFormer: Revisiting the Roles of Sequence Modeling and Feature Interaction in CTR Prediction. *arXiv preprint arXiv:2601.12681* (2026).
- [12] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General perception with iterative attention. In *International conference on machine learning*. PMLR, 4651–4664.
- [13] Shali Jiang, Hua Zheng, Boyang Liu, Laming Chen, Kenny Lov, Chuanqi Xu, Lisang Ding, Qinghai Zhou, Can Cui, Xiaolong Liu, et al. 2026. LoopFM: Learning from Historical Representations of Foundation Model for Recommendation. *arXiv preprint arXiv:2605.29280* (2026).
- [14] Nikhil Khani, Li Wei, Aniruddh Nath, Shawn Andrews, Shuo Yang, Yang Liu, Pendo Abbo, Maciej Kula, Jarrod Kahn, Zhe Zhao, et al. 2024. Bridging the gap: Unpacking the hidden challenges in knowledge distillation for online ranking systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 758–761.
- [15] Weijiang Lai, Beihong Jin, Jiongyan Zhang, Yiyuan Zheng, Jian Dong, Jia Cheng, Jun Lei, and Xingxing Wang. 2025. Exploring Scaling Laws of CTR Model for Online Performance Improvement. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 114–123.
- [16] Kaiyuan Li, Yongxiang Tang, Yanhua Cheng, Yong Bai, Yanxiang Zeng, Chao Wang, Xialong Liu, and Peng Jiang. 2025. VQL: An End-to-End Context-Aware Vector Quantization Attention for Ultra-Long User Behavior Modeling. *arXiv preprint arXiv:2508.17125* (2025).
- [17] Xinchun Li, Ning Zhang, Qianqian Yang, Fei Teng, Wenlin Zhao, Huizhi Yang, Heng Shi, Linlan Chen, Yixin Wu, Zhen Wang, et al. 2026. IAT: Instance-As-Token Compression for Historical User Sequence Modeling in Industrial Recommender Systems. *arXiv preprint arXiv:2604.08933* (2026).
- [18] Mingyang Liu, Yong Bai, Zhangming Chan, Sishuo Chen, Xiang-Rong Sheng, Han Zhu, Jian Xu, and Xinyang Chen. 2026. EST: Towards Efficient Scaling Laws in Click-Through Rate Prediction via Unified Modeling. *arXiv preprint arXiv:2602.10811* (2026).
- [19] Zhiwei Liu, Ziwei Fan, Yu Wang, and Philip S Yu. 2021. Augmenting sequential recommendation with pseudo-prior items via reversely pre-training transformer. In *Proceedings of the 44th international ACM SIGIR conference on Research and development in information retrieval*. 1608–1612.
- [20] Xiao Lv, Jiangxia Cao, Shijie Guan, Xiaoyou Zhou, Zhiguang Qi, Yaqiang Zang, Ben Wang, and Guorui Zhou. 2025. MARM: Unlocking the Recommendation Cache Scaling-Law through Memory Augmentation and Scalable Complexity. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2022–2031.
- [21] Wenhan Lyu, Devashish Tyagi, Yihang Yang, Ziwei Li, Ajay Somani, Karthikeyan Shanmugasundaram, Nikola Andrejevic, Ferdi Adeputra, Curtis Zeng, Arun K Singh, et al. 2025. DV365: Extremely Long User History Modeling at Instagram. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4717–4727.
- [22] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.
- [23] Ads Recommendation. 2025. External Large Foundation Model: How to Efficiently Serve Trillions of Parameters for Online Ads Recommendation. *arXiv preprint arXiv:2502.17494* (2025).
- [24] Zihua Si, Lin Guan, ZhongXiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, et al. 2024. Twin v2: Scaling ultra-long user behavior sequence modeling for enhanced ctr prediction at kuaishou. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4890–4897.
- [25] Xin Song, Zhilin Guan, Ruidong Han, Binghao Tang, Tianwen Chen, Bing Li, Zihao Li, Han Zhang, Fei Jiang, Qing Wang, et al. 2026. MTFM: A Scalable and Alignment-free Foundation Model for Industrial Recommendation in Meituan. *arXiv preprint arXiv:2602.11235* (2026).
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [27] Jesse Vig and Yonatan Belinkov. 2019. Analyzing the structure of attention in a transformer language model. In *Proceedings of the 2019 ACL workshop Black-box NLP: analyzing and interpreting neural networks for NLP*. 63–76.
- [28] Shuli Wang. 2026. Sample Is Feature: Beyond Item-Level, Toward Sample-Level Tokens for Unified Large Recommender Models. *arXiv preprint arXiv:2604.15650* (2026).
- [29] Zhuoxing Wei, Qi Liu, and Qingchen Xie. 2025. Deep Multiple Quantization Network on Long Behavior Sequence for Click-Through Rate Prediction. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3090–3094.
- [30] Xue Xia, Saurabh Joshi, Kousik Rajesh, Kangan Li, Yangyi Lu, Nikil Pancha, Dhruvil Badani, Jiajing Xu, and Pong Eksombatchai. 2025. TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6881–6882.
- [31] Lei Xin, Yuhao Zheng, Ke Cheng, Changjiang Jiang, Zifan Zhang, and Fanhu Zeng. 2026. Hytrec: A hybrid temporal-aware attention architecture for long behavior sequential recommendation. *arXiv preprint arXiv:2602.18283* (2026).
- [32] Lee Xiong, Zhirong Chen, Rahul Mayuranath, Shangran Qiu, Arda Ozdemir, Lu Li, Yang Hu, Dave Li, Jingtao Ren, Howard Cheng, et al. 2026. LLaTTE: Scaling Laws for Multi-Stage Sequence Modeling in Large-Scale Ads Recommendation. *arXiv preprint arXiv:2601.20083* (2026).
- [33] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. 2025. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23078–23097.
- [34] Kun Yuan, Junyu Bi, Daixuan Cheng, Changfa Wu, Shuwen Xiao, Binbin Cao, Jian Wu, and Yuning Jiang. 2026. HiSAC: Hierarchical Sparse Activation Compression for Ultra-long Sequence Modeling in Recommenders. *arXiv preprint arXiv:2602.21009* (2026).
- [35] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [36] Shangyu Zhang, Shijie Quan, Zhongren Wang, Junwei Pan, Tianqu Zhuang, Bo Fu, Yilong Sun, Jieying Lin, Jushuo Chen, Xiaotian Li, et al. 2025. Large Foundation Model for Ads Recommendation. *arXiv preprint arXiv:2508.14948* (2025).
- [37] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.
- [38] Wen-Ji Zhou, Yuhang Zheng, Yinfu Feng, Yunan Ye, Rong Xiao, Long Chen, Xiaosong Yang, and Jun Xiao. 2024. ENCODE: Breaking the trade-off between performance and efficiency in long-term user behavior modeling. *IEEE Transactions on Knowledge and Data Engineering* 37, 1 (2024), 265–277.
- [39] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.